Guia Prático de Git e GitHub

1. Configuração Inicial

Execute estes comandos apenas uma vez após instalar o Git na sua máquina, para que o sistema saiba quem está fazendo as alterações:

- git config --global user.name "Seu Nome": Define o nome que aparecerá no histórico.
- git config --global user.email "seuemail@exemplo.com": Define o e-mail atrelado aos seus commits.

2. Navegação Básica no Terminal (Bash)

Antes de usar o Git, você precisa saber se mover pelas pastas do seu computador usando linhas de comando:

- pwd: Mostra o caminho completo da pasta (diretório) em que você está no momento.
- ls: Lista todos os arquivos e pastas do diretório atual.
- cd nomedapasta: Entra em uma pasta específica. *(use cd .. para voltar uma pasta).*
- clear: Limpa a tela do terminal para facilitar a leitura.
- exit: Encerra o terminal com segurança.

3. Iniciando seu Projeto

Comandos para criar o vínculo entre a sua máquina e o GitHub:

- git clone https://github.com/usuario/repositorio.git: Copia um repositório existente no GitHub para a sua máquina local, criando a pasta automaticamente.
- git init: Transforma a pasta atual da sua máquina em um repositório Git. *(Use quando for começar um projeto do zero localmente).*

4. Fluxo de Trabalho

Esta é a parte na qual você estará revisitando este documento a maioria das vezes para salvar e enviar seu código:

- git status: Mostra o estado atual do repositório (arquivos modificados, adicionados ou com erros). Use o tempo todo para verificações.
- git add nome_do_arquivo.ext: Adiciona um arquivo específico à área de preparação (chamamos de *Staging Area*), avisando ao Git que ele fará parte do próximo salvamento.
- git add . : Adiciona **todos** os arquivos modificados e novos da pasta atual de uma só vez.
- git commit -m "Mensagem clara do que foi feito": Salva um "retrato" das alterações

preparadas pelo comando add. A mensagem deve ser curta e explicar a mudança (verbos no imperativo).

- git push origin main: Envia os seus commits da máquina local para a nuvem no GitHub. *(projetos mais antigos podem usar a nomenclatura master em vez de main).*
- git pull origin main: Puxa as atualizações que estão no GitHub para a sua máquina. **Sempre faça isso antes de começar a trabalhar** se estiver em uma equipe!!!

5. Branches (Ramificações/camadas)

Use branches para criar novas funcionalidades sem correr o risco de quebrar o código principal (a branch main):

- git branch: Lista todas as branches locais do seu projeto.
- git checkout -b nome-da-nova-branch: Cria uma nova ramificação isolada e já muda para ela.
- git checkout main: Retorna para a branch principal.

6. Histórico e Correções

Consulte etapas passadas e desfaça ações quando necessário:

- git log: Exibe o histórico de todos os commits feitos (autor, data e o código *hash*, ex: a1b2c3d). *Pressione 'q' para sair desta tela.*
- git show <hash-do-commit>: Mostra os detalhes exatos e as linhas alteradas em um commit específico.
- git restore <arquivo>: Desfaz as modificações em um arquivo que **ainda não foi commitado**, voltando ao estado do último salvamento.
- git revert <hash-do-commit>: Cria um novo commit que anula as alterações de um commit anterior. É a forma mais segura de corrigir erros em equipe.
- git reset --hard <hash-do-commit>: **Cuidado!** Volta o repositório no tempo de forma definitiva, apagando permanentemente as alterações feitas após o commit especificado.